

PROJECT-BASED EXAM

Course: Natural Language Processing

To complete this project based final exam, **only you can do one of the following options (select only one option!)**:

I - Option 1: Predefined Project

Binary Text Classification

1. Topic

Toxic Comment Identification

2. Goal

The objective of this project is to develop a binary text classification system capable of distinguishing between 'Toxic' and 'Non-Toxic' comments using modern Natural Language Processing (NLP) techniques.

3. Dataset Details

Students must use the following dataset from Kaggle:

<https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge/data?select=train.csv.zip>

The dataset originally contains multiple toxicity labels. These must be converted into a binary classification problem as follows:

- Label 1 (Toxic): If any of the toxicity-related labels is equal to 1
- Label 0 (Non-Toxic): If all labels are equal to 0

4. General Instructions

- The project must be implemented using Python

- Preferred platforms: Kaggle, Google Colab, or Jupyter Notebook
- Notebook name format: Name_Surname_NLP_Midterm
- The final submission must be publicly accessible.

5. Data Preprocessing

The following preprocessing steps are mandatory before model training:

- Convert all text to lowercase.
- Remove punctuation, special characters, and numbers.
- Remove English stop words
- Split dataset into the train and test parts.
- Apply tokenization (required for deep learning models)

6. Model Development

Students complete the project described in this document using the required models and evaluation steps.

6.1 Model 1: Deep Learning with Dynamic BERT Embeddings

- Develop a custom Deep Learning model
- Integrate contextual embeddings using BERT (e.g., bert-base-uncased)
- The architecture must include at least three layers such as: Dense, Dropout, LSTM, BiLSTM, Conv1D, BatchNormalization, or Pooling layers
- Train and evaluate the model on the on the training and test datasets

6.2 Model 2: Hugging Face DistilBERT Transformer model using HF Pipeline or AutoTokenizer and AutoModel

- Use Hugging Face's text-classification pipeline
- Use a DistilBERT-based transformer model (base or fine-tuned)
- Evaluate the model performance on the test dataset

6.3 Model 3: LLaMA 3 LLM Model from the Hugging Face Hub or using other API

- Use one of the following LLaMA 3 variants: 1B, 3B, or 8B (this model require more poerful computer)
- Perform classification using prompting (zero-shot or few-shot) or fine-tuning

- Evaluate the model performance on the test dataset

6.4 Evaluation Criteria

- Accuracy (Testing only) for all models
- Confusion Matrix for all models

6.5 Results Table

Model	Description	Testing Accuracy
Model 1	DL + BERT	
Model 2	DistilBERT Transformer model with HF Pipeline	
Model 3	LLaMA 3 LLM with HF API or other API	

II - Option 2: Custom Project

Students may propose their own project, provided that:

- The project includes at least one NLP task.
- Project must have either a transformer or a LLM model.
- It may be extended with ML, DL, or other AI techniques.
- **The problem, methodology, and evaluation must be clearly defined at the beginning of the Notebook in the markdown cell.**

Final Deliverables

- **Only** submit a working Python Notebook **link** (Kaggle, Colab, or GitHub).
- Ensure the notebook or repository is publicly accessible
- **Submit the project link via Moodle**
- **Attention:** If both options are submitted, only Option 1 will be evaluated.

For custom projects: if your own proposed NLP project is involving web application and mobile application or any UI

- Upload full source code to GitHub.
- Include screenshots demonstrating that the application is fully functional running.